

Mizar Evo **の設計指針について**  
**自動証明・数学的記述・検証可能な計算の再接続**  
*presenter notes edition*

**中正 和久**

岩手県立大学

TPP 2026、**理化学研究所 AIP 東京オフィス**、2026 年 11 月 16–17 日

## Mizar Evo の設計指針について

Mizar Evo の設計指針について  
自動証明・数学的記述・検証可能な計算の再接続  
*presenter notes edition*

中正 和久

岩手県立大学

TPP 2026、理化学研究所 AIP 東京オフィス、2026 年 11 月 16-17 日

### Read aloud:

- Mizar は証明検査系です。一階の集合論の上に、人間が読める数学の言葉で書かれた 50 年分のライブラリを持っています。
- Mizar Evolution、略して Mizar Evo は、その言語と道具を設計し直すプロジェクトです。現行 Mizar の課題を整理し、新仕様でどのように対応するかを説明します。
- 話す内容にはすべて、事実・研究仮説・将来構想のラベルを付けます。

## 0.3 - 現行 Mizar の課題と 6 つの設計指針

§ 1 基盤:

§ 2 記述:

§ 3 汎用化:

§ 4 計算:

§ 5 開発基盤:

§ 6 検査・自動化:

# Mizar Evo の設計指針について

## └ Part 0. Introduction

### └ 0.3 - 現行 Mizar の課題と 6 つの設計指針

#### 0.3 - 現行 Mizar の課題と 6 つの設計指針

| 現行 Mizar から取り組む課題           | 新仕様の設計指針                           |
|-----------------------------|------------------------------------|
| § 1 基盤: 論理・MML の継承と処理系の刷新   | 一階論理・集合論を維持し、kernel を小さく保つ         |
| § 2 記述: 暗黙の型・登録・演算選択の追跡     | 数学的な抽象化を継承し、暗黙の選択を明示               |
| § 3 汎用化: 定義・定理・scheme の個別機構 | template で共通化の仕組みを統一               |
| § 4 計算: 証明と実行可能な手続きの接続      | algorithm の契約・不変条件・停止性を検査          |
| § 5 開発基盤: 依存管理・配布・差分検証・IDE  | module・namespace・package・差分ビルド・LSP |
| § 6 検査・自動化: 外部探索と検査根拠の接続    | ATP を活用し、evidence のインスタンス化と SAT 検査 |

Optional detail (see slide).

### Optional notes and sources:

- 課題と設計指針を同じ行に対応させる。以下の § 1 - § 6 が各行に対応。一階の基盤と可読な記述は、継承すべき強み。
- 新仕様と実装済みの範囲は、末尾の「仕様と実装の状況」で区別する。
- Source: 仕様 01, 18, 20, 21, 23; Białystok 資料の課題別ストーリー。

## 1.1 - 基盤の継承: 論理は保ち、処理系を再整備

数学的記述の層を継承し、  
処理系と開発基盤を再構築する。

- 一階論理と Tarski-Grothendieck 集合論を基盤として維持。
- soft type、mode、attribute、registration、structure、**宣言的証明を継承。**
- **証明探索と検査を分離し、受理を小規模な kernel に集約。詳細は § 6。**

# Mizar Evo の設計指針について

## └ Part 1. Logical Foundation

### └ 1.1 - 基盤の継承: 論理は保ち、処理系を再整備

1.1 - 基盤の継承: 論理は保ち、処理系を再整備

数学的記述の層を継承し、  
処理系と開発基盤を再構築する。

- 一階論理と Tarski-Grothendieck 集合論を基盤として維持。
- soft type、mode、attribute、registration、structure、宣言的証明を継承。
- 証明探索と検査を分離し、受理を小規模な kernel に集約。詳細は § 6。

### Read aloud:

- 数学的記述の層を継承し、処理系と開発基盤を再構築する。
- 一階論理と Tarski-Grothendieck 集合論を基盤として維持。
- soft type、mode、attribute、registration、structure、宣言的証明を継承。
- 証明探索と検査を分離し、受理を小規模な kernel に集約。詳細は § 6。

### Optional notes and sources:

- 数学的な記述と証明支援に必要な機能は、一階の基盤の上にも構成できる。「仕組みが近い」はこれらの構成を指し、論理体系や表現方法の同一性を意味しない。
- 成功率や再構成の失敗率から一階の性能優位性を主張しない。設計の効果は、記述の利便性・自動証明・証明検査の観点で評価する。

## 1.2 - 現行 Mizar: 集合論的な関数

exact MML excerpt

```
func f.x -> set means  
:Def2:  
[x,it] in f if x in dom f otherwise it = {};
```

exact MML excerpt

```
definition  
  let X,Y;  
  mode Function of X,Y is quasi_total PartFunc of X,Y;  
end;
```

- 関数是对の集合として表される一階の対象。関数適用は集合論上で定義する。
- 関数の扱いに、基盤論理の高階化は不要。
- 記述上の課題: 定義域・グラフ・関数性・所属関係を直接記述すると煩雑。

## Mizar Evo の設計指針について

## └ Part 1. Logical Foundation

## └ 1.2 - 現行 Mizar: 集合論的な関数

## 1.2 - 現行 Mizar: 集合論的な関数

関数適用は定義された集合論的關係 exact MML excerpt

```
func f.x -> set means
:Def2:
[x,it] in f if x in dom f otherwise it = {};
```

型付き関数は部分関数の上の soft type exact MML excerpt

```
definition
let X,Y;
mode Function of X,Y is quasi_total PartFunc of X,Y;
end;
```

- 関数は対の集合として表される一階の対象。関数適用は集合論上で定義する。
- 関数の扱いに、基盤論理の高階化は不要。
- 記述上の課題: 定義域・グラフ・関数性・所属関係を直接記述すると煩雑。

## Read aloud:

- 関数適用は定義された集合論的關係。
- 型付き関数は部分関数の上の soft type.
- 関数は対の集合として表される一階の対象。関数適用は集合論上で定義する。
- 関数の扱いに、基盤論理の高階化は不要。
- 記述上の課題: 定義域・グラフ・関数性・所属関係を直接記述すると煩雑。

## Optional notes and sources:

- Source (2026 年 10 月 7 日確認) :  
[https://mizar.uwb.edu.pl/version/current/mml/funct\\_1.miz](https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz) (138-140 行、  
 FUNCT\_1:def 2)、[https://mizar.uwb.edu.pl/version/current/mml/funct\\_2.miz](https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz)  
 (87-90 行)。GPL-3.0-or-later / CC-BY-SA-3.0-or-later。
- quasi\_total (FUNCT\_2 の 36-44 行) は、Y が空でなければ定義域が X 全体であることを言う。

## 2.1 - 記述の課題: 可読性と抽象化を継承・拡張

sketch

```
let X, Y be set;  
let f be Function of X, Y;  
let x be Element of X;  
... f.x ...
```

- 集合論的な関数を `Function of X,Y` と `f.x` で記述し、対や定義域の詳細を抽象化する。
- `soft type`、`mode`、`attribute`、`registration`、`scheme`、宣言的証明は、単なる記法の簡略化を超え、一階の集合論を可読な数学的記述へ結び付ける言語機構。
- 新仕様: 数学的な記述を継承・拡張し、型・登録・オーバーロードの暗黙の選択を明示・追跡可能にする。

# Mizar Evo の設計指針について

## └ Part 2. Readable Mathematics

### └ 2.1 - 記述の課題: 可読性と抽象化を継承・拡張

#### Read aloud:

- ソース上の関数表現.
- 集合論的な関数を `Function of X,Y` と `f.x` で記述し、対や定義域の詳細を抽象化する。
- `soft type`、`mode`、`attribute`、`registration`、`scheme`、宣言的証明は、単なる記法の簡略化を超え、一階の集合論を可読な数学的記述へ結び付ける言語機構。
- 新仕様: 数学的な記述を継承・拡張し、型・登録・オーバーロードの暗黙の選択を明示・追跡可能にする。

#### 2.1 - 記述の課題: 可読性と抽象化を継承・拡張

ソース上の関数表現 **sketch** (現行 Mizar と Mizar Evo で有効)

```
let X, Y be set;  
let f be Function of X, Y;  
let x be Element of X;  
... f.x ...
```

- 集合論的な関数を `Function of X,Y` と `f.x` で記述し、対や定義域の詳細を抽象化する。
- `soft type`、`mode`、`attribute`、`registration`、`scheme`、宣言的証明は、単なる記法の簡略化を超え、一階の集合論を可読な数学的記述へ結び付ける言語機構。
- 新仕様: 数学的な記述を継承・拡張し、型・登録・オーバーロードの暗黙の選択を明示・追跡可能にする。

## 2.2 - 暗黙の演算選択を明示・追跡する

sketch

```
let R be commutative Ring;  
f(R);                :: ambiguous Magma view  
f(R qua AddMagma);   :: use addition  
f(R qua MulMagma);   :: use multiplication
```

- 課題: 環は加法・乗法の両方で Magma を継承。f(R) だけではビューが定まらない。
- 新仕様: qua で加法・乗法のビューを明示し、曖昧な選択を解消。
- 自動適用した登録は適用経路を記録し、依存する規則を追跡可能にする。

## Mizar Evo の設計指針について

## └ Part 2. Readable Mathematics

## └ 2.2 - 暗黙の演算選択を明示・追跡する

## 2.2 - 暗黙の演算選択を明示・追跡する

環の二つの演算から、利用するビューを選択 [sketch](#)

```

let R be commutative Ring;
f(R);
f(R qua AddMagma);
f(R qua MulMagma);

```

*// ambiguous Magma view*  
*// use addition*  
*// use multiplication*

- 課題: 環は加法・乗法の両方で Magma を継承。f(R) だけではビューが定まらない。
- 新仕様: qua で加法・乗法のビューを明示し、曖昧な選択を解消。
- 自動適用した登録は適用経路を記録し、依存する規則を追跡可能にする。

## Read aloud:

- 環の二つの演算から、利用するビューを選択。
- 課題: 環は加法・乗法の両方で Magma を継承。f(R) だけではビューが定まらない。
- 新仕様: qua で加法・乗法のビューを明示し、曖昧な選択を解消。
- 自動適用した登録は適用経路を記録し、依存する規則を追跡可能にする。

## Optional notes and sources:

- f は Magma の演算を使う functor の概形。必要な定義・継承・登録を前提とした呼出し例。
- qua は既存 Mizar にもある記法。ここでは新仕様の継承パスの明示と、曖昧性の説明を示す。
- Source: doc/spec/en/19.overload\_resolution.md § 19.3.1; 17.clusters\_and\_registrations.md, Traceability。

## 3.1 - Template: functor の定義と結果型を共通化

例: 関数の和。「各点で値を足す」という構成を共通化。

sketch

```
definition
  let T be type extends non empty AddMagma;
  let I be non empty set;
  let f, g be Function of I, T;
  func AddDef: Add[T,I](f,g) -> Function of I,T means
    for i being Element of I holds it.i = T.add(f.i,g.i);
end;
```

- 現行 MML: 実数値・複素数値ごとに、和の結果型の再定義・登録が付随。
- 新仕様: 値の型  $T$  をパラメータ化し、同じ構成の定義と結果型を共通化。

## Mizar Evo の設計指針について

## └ Part 3. Generic Mathematics

## └ 3.1 - Template: functor の定義と結果型を共通化

## 3.1 - Template: functor の定義と結果型を共通化

例: 関数の和。「各点で値を足す」という構成を共通化。

汎用 functor `sketch` (存在・一意性の証明は省略)

```

definition
  let T be type extends non empty AddMagma;
  let I be non empty set;
  let f, g be Function of I, T;
  func AddDef: Add[I,I](f,g) -> Function of I,T means
    for i being Element of I holds it.i = T.add(f.i,g.i);
end;

```

- 現行 MML: 実数値・複素数値ごとに、和の結果型の再定義・登録が付随。
- 新仕様: 値の型  $T$  をパラメータ化し、同じ構成の定義と結果型を共通化。

## Read aloud:

- 例: 関数の和。「各点で値を足す」という構成を共通化。
- 汎用 functor.
- 現行 MML: 実数値・複素数値ごとに、和の結果型の再定義・登録が付随。
- 新仕様: 値の型  $T$  をパラメータ化し、同じ構成の定義と結果型を共通化。

## Optional notes and sources:

- $T$  は加法を持つ型、 $I$  は共通の非空添字集合。必要な構造・登録を前提に、本体を一度検査して具体化する。共通化の対象は同じ構成の定義族。
- Source: doc/spec/en/18.templates.md § 18.2.2, 18.7, 18.10.1; sample\_codes.md AddMagma; MML VALUED\_1  
([https://mizar.uwb.edu.pl/version/current/html/valued\\_1.html](https://mizar.uwb.edu.pl/version/current/html/valued_1.html))。

## specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

- 述語パラメータにより、代入する述語ごとの一階定理の族を表現する。
- 明示的なインスタンス化: defpred に続けて by NatInduction[P], Base, Step。
- 関数子パラメータは集合ではなく schema レベルの記号として扱い、一階論理を維持する。
- 詳細: Białystok 資料 Story 6 (PermProduct[T]、qua によるビュー)、および本資料 Backup 4。

## Mizar Evo の設計指針について

## └ Part 3. Generic Mathematics

## └ 3.2 - Scheme を template に統合

deep dive

3.2 - Scheme を template に統合

deep dive

述語パラメータ付き定理としての帰納法

specification example

```

definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;

```

- 述語パラメータにより、代入する述語ごとの一階定理の族を表現する。
- 明示的なインスタンス化: defpred に続けて by NatInduction[P], Base, Step。
- 関数子パラメータは集合ではなく schema レベルの記号として扱い、一階論理を維持する。
- 詳細: Białystok 資料 Story 6 (PermProduct [T], qua によるビュー)、および本資料 Backup 4。

## Read aloud:

- 述語パラメータ付き定理としての帰納法。
- 述語パラメータにより、代入する述語ごとの一階定理の族を表現する。
- 明示的なインスタンス化: defpred に続けて by NatInduction[P], Base, Step。
- 関数子パラメータは集合ではなく schema レベルの記号として扱い、一階論理を維持する。
- 詳細: Białystok 資料 Story 6 (PermProduct [T]、qua によるビュー)、および本資料 Backup 4。

## 4.1 - 計算の課題: 検証可能な algorithm

- 現行 Mizar は宣言的証明と組込みの自動化を採用。ユーザ定義の tactic 言語は持たない。
- 課題: 数学的な証明と実行可能な計算を、一つの検査の枠組みに結び付ける。
- 新仕様: algorithm は契約を持つ手続き。正しさ・不変条件・停止性を検査する。
- 用途: アルゴリズム自体の検証と、検証済み手続きによる証明支援。

# Mizar Evo の設計指針について

## └ Part 4. Verified Computation

### └ 4.1 - 計算の課題: 検証可能な algorithm

#### 4.1 - 計算の課題: 検証可能な algorithm

- 現行 Mizar は宣言的証明と組込みの自動化を採用。ユーザ定義の tactic 言語は持たない。
- 課題: 数学的な証明と実行可能な計算を、一つの検査の枠組みに結び付ける。
- 新仕様: algorithm は契約を持つ手続き。正しさ・不変条件・停止性を検査する。
- 用途: アルゴリズム自体の検証と、検証済み手続きによる証明支援。

### Read aloud:

- 現行 Mizar は宣言的証明と組込みの自動化を採用。ユーザ定義の tactic 言語は持たない。
- 課題: 数学的な証明と実行可能な計算を、一つの検査の枠組みに結び付ける。
- 新仕様: algorithm は契約を持つ手続き。正しさ・不変条件・停止性を検査する。
- 用途: アルゴリズム自体の検証と、検証済み手続きによる証明支援。

### Optional notes and sources:

- Source: `doc/spec/en/20.algorithm_and_verification.md`, sections 20.1 and 20.12.

## 4.2 - Algorithm: 契約、証明、計算

### 契約付きのユークリッドの互除法 specification example

```
terminating algorithm euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a; var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y); decreasing y;
    const r := x mod y; x := y; y := r;
  end;
  return x;
end;
```

- 契約・不変条件・停止性の測度から一階の証明義務を生成し、定理と同じ枠組みで検査する。
- 二つの役割: 検証済みの自動化手続きと、by computation による検査済みの計算。仕様化済み。MVM 実行とコード抽出は今後の課題。

## Mizar Evo の設計指針について

## └ Part 4. Verified Computation

## └ 4.2 - Algorithm: 契約、証明、計算

## 4.2 - Algorithm: 契約、証明、計算

## 契約付きのユークリッドの互除法

specification example

```

terminating algorithm euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a; var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y); decreasing y;
    const r := x mod y; x := y; y := r;
  end;
  return x;
end;

```

- 契約・不変条件・停止性の測度から一階の証明義務を生成し、定理と同じ枠組みで検査する。
- 二つの役割: 検証済みの自動化手続きと、by computation による検査済みの計算。仕様化済み。MVM 実行とコード抽出は今後の課題。

## Read aloud:

- 契約付きのユークリッドの互除法.
- 契約・不変条件・停止性の測度から一階の証明義務を生成し、定理と同じ枠組みで検査する。
- 二つの役割: 検証済みの自動化手続きと、by computation による検査済みの計算。仕様化済み。MVM 実行とコード抽出は今後の課題。

## Optional notes and sources:

- Source: doc/spec/en/20.algorithm\_and\_verification.md, section 20.12 (外側の definition ブロックと let a, b be Nat; を省略)。

- 整数論・組合せ論のアルゴリズム、記号計算、最適化手続き。
- 長期的な対象: 暗号アルゴリズムとプロトコル、量子アルゴリズムと古典・量子ハイブリッド。
- algorithm は、証明の自動化手続きと応用アルゴリズムの検証を共通の枠組みに結び付ける。対象の拡大は将来構想。

## Mizar Evo の設計指針について

## └ Part 4. Verified Computation

## └ 4.3 - Algorithm の応用範囲（将来構想）

deep dive

## 4.3 - Algorithm の応用範囲（将来構想）

deep dive

将来構想（現在の対応範囲には含まれない）：

- 整数論・組合せ論のアルゴリズム、記号計算、最適化手続き。
- 長期的な対象：暗号アルゴリズムとプロトコル、量子アルゴリズムと古典・量子ハイブリッド。

共通する検証・実行の流れ：手続き・契約の記述 → 不変条件・停止性の指定 → 証明義務の生成 → ATP による証明と kernel 検査 → 具体的な入力での実行 → コード抽出（将来）。

- **algorithm** は、証明の自動化手続きと応用アルゴリズムの検証を共通の枠組みに結び付ける。対象の拡大は将来構想。

## Read aloud:

- 整数論・組合せ論のアルゴリズム、記号計算、最適化手続き。
- 長期的な対象：暗号アルゴリズムとプロトコル、量子アルゴリズムと古典・量子ハイブリッド。
- 手続き・契約の記述 → 不変条件・停止性の指定 → 証明義務の生成 → ATP による証明と kernel 検査 → 具体的な入力での実行 → コード抽出（将来）。
- **algorithm** は、証明の自動化手続きと応用アルゴリズムの検証を共通の枠組みに結び付ける。対象の拡大は将来構想。

## 5.1 - 開発基盤の課題と新仕様の対応

- Mizar の数学的思想を継承し、ソフトウェア構成は現代の開発要件に合わせて再設計する。
- 言語とともに開発基盤全体を現代化。各機能の詳細は Białystok 資料を参照。

# Mizar Evo の設計指針について

## └ Part 5. Development Infrastructure

### └ 5.1 - 開発基盤の課題と新仕様の対応

#### 5.1 - 開発基盤の課題と新仕様の対応

| MML の 50 年の蓄積から見えた課題 | Mizar Evo                              |
|----------------------|----------------------------------------|
| article が依存の単位       | 明示的 import を持つモジュール                    |
| グローバルな名前管理           | namespace、完全修飾名                        |
| 配布とバージョン管理が弱い        | package、SemVer、lock file               |
| フルビルド                | 依存指紋による差分ビルド                           |
| ATP は外付け、自動化が見えにくい   | 第一級の ATP パイプライン、解決トレース、kernel evidence |
| IDE 連携と機械可読な入出力が弱い   | LSP、構造化診断、エージェント向けインタフェース              |

- Mizar の数学的思想を継承し、ソフトウェア構成は現代の開発要件に合わせて再設計する。
- 言語とともに開発基盤全体を現代化。各機能の詳細は Białystok 資料を参照。

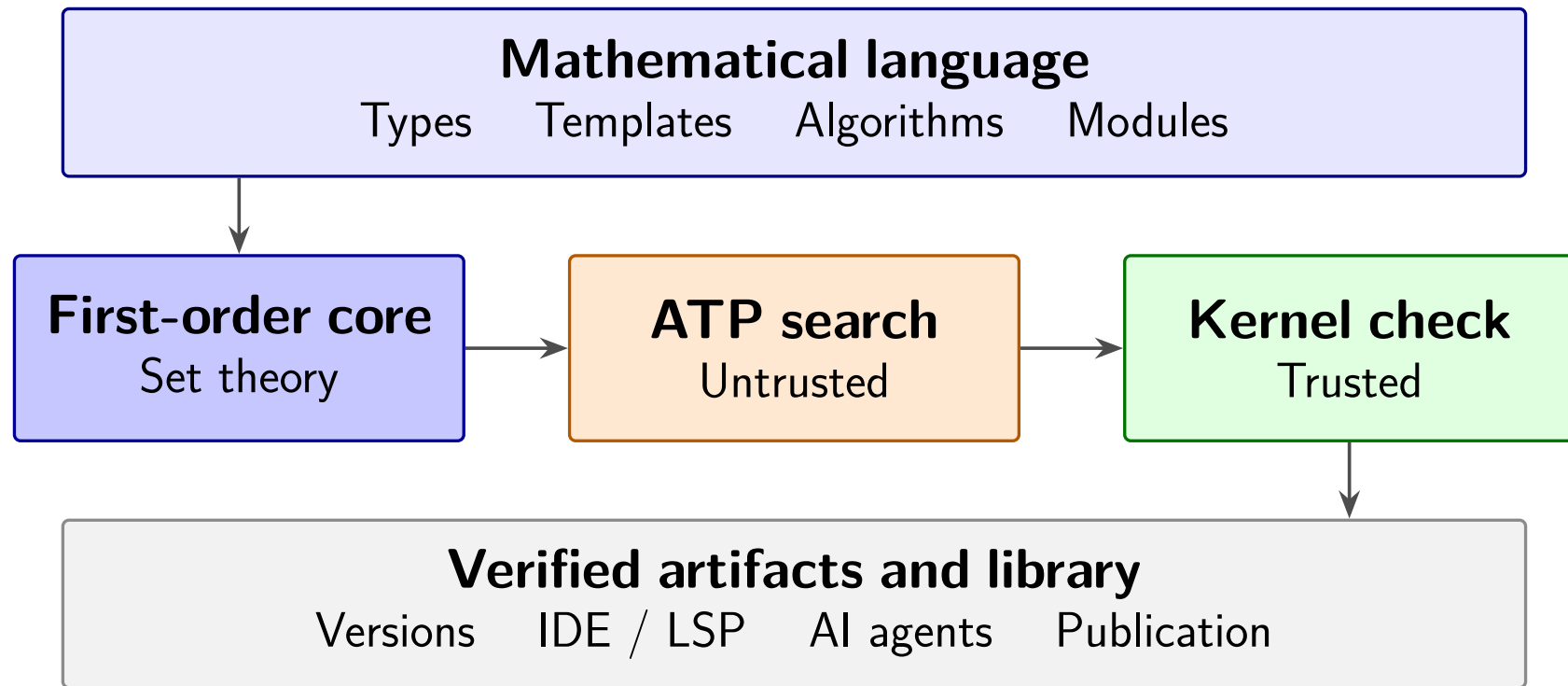
### Read aloud:

- Mizar の数学的思想を継承し、ソフトウェア構成は現代の開発要件に合わせて再設計する。
- 言語とともに開発基盤全体を現代化。各機能の詳細は Białystok 資料を参照。

### Optional notes and sources:

- 批判ではない。50 年前には一般的でなかったソフトウェア工学を、形式数学の環境に持ち込む話。
- 詳細: Białystok 資料 Story 1, 3, 5, 8; 本資料 Backup 6-8。

## 5.2 - 全体像



- 豊かな数学的記述を、小規模な一階論理の基盤と現代的な開発環境で支える。

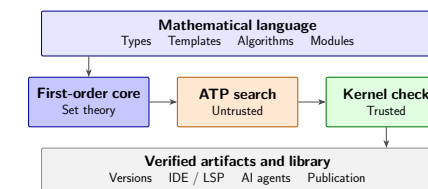
2026-10-09

# Mizar Evo の設計指針について

## └ Part 5. Development Infrastructure

### └ 5.2 - 全体像

#### 5.2 - 全体像



• 豊かな数学的記述を、小規模な一階論理の基盤と現代的な開発環境で支える。

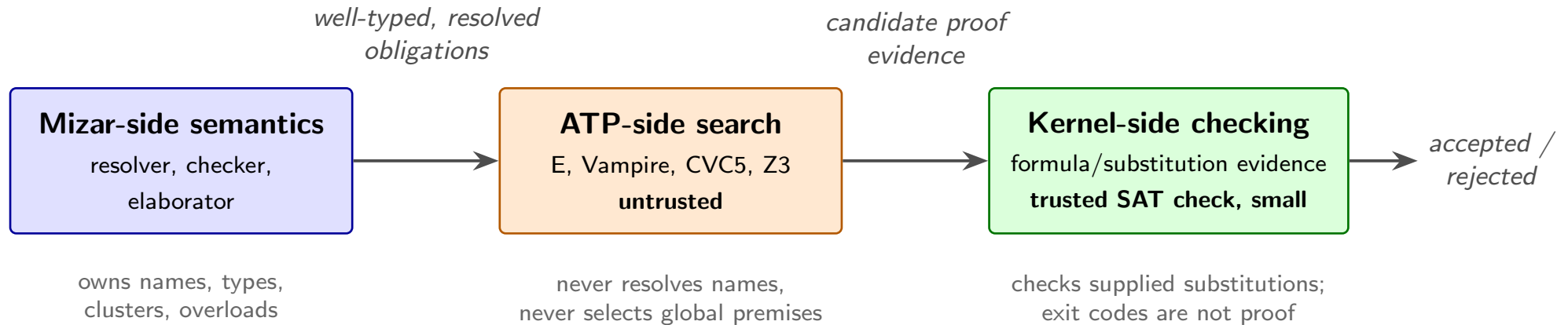
### Read aloud:

- 豊かな数学的記述を、小規模な一階論理の基盤と現代的な開発環境で支える。

### Optional notes and sources:

- 数学的な言語は elaboration を経て一階表現へ移る。外部 ATP の候補 evidence を kernel で検査し、検証済み成果物・ライブラリを IDE / LSP、AI、出版に利用する。

## 6.1 - 自動化の課題: 証明探索と検査の分離



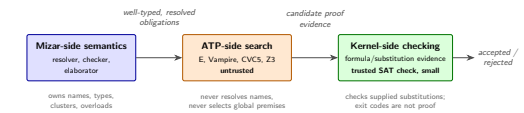
- 一階 ATP は証明探索を担う。探索結果は、信頼できる検査器による検証を要する。
- Mizar 側: 名前・型・cluster・オーバーロードの解決。ATP: 証明探索。kernel: 論理式と代入を、小規模な信頼できる SAT 検査で検証し、受理を判定。
- ATP の終了コードのみでは証明を受理しない。一階自動推論の導入によって信頼基盤を拡大しない設計。

## Mizar Evo の設計指針について

## └ Part 6. Checking And Automation

## └ 6.1 - 自動化の課題: 証明探索と検査の分離

## 6.1 - 自動化の課題: 証明探索と検査の分離



- 一階 ATP は証明探索を担う。探索結果は、信頼できる検査器による検証を要する。
- Mizar 側: 名前・型・cluster・オーバーロードの解決。ATP: 証明探索。kernel: 論理式と代入を、小規模な信頼できる SAT 検査で検証し、受理を判定。
- ATP の終了コードのみでは証明を受理しない。一階自動推論の導入によって信頼基盤を拡大しない設計。

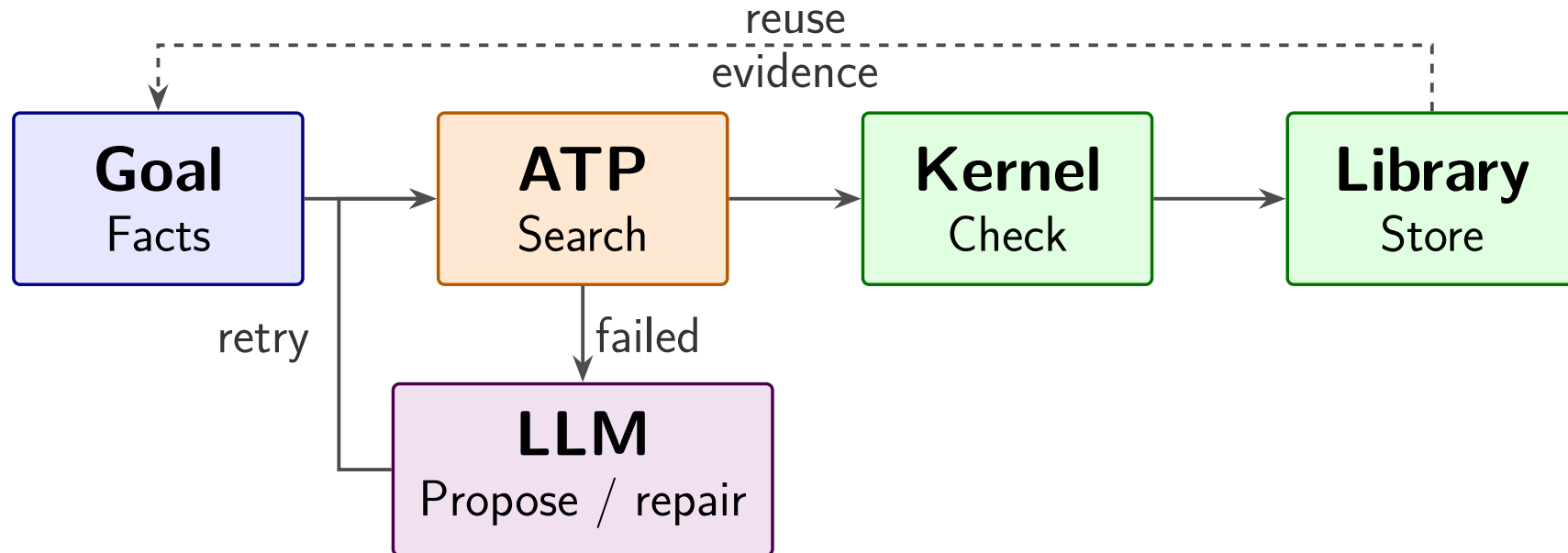
## Read aloud:

- 一階 ATP は証明探索を担う。探索結果は、信頼できる検査器による検証を要する。
- Mizar 側: 名前・型・cluster・オーバーロードの解決。ATP: 証明探索。kernel: 論理式と代入を、小規模な信頼できる SAT 検査で検証し、受理を判定。
- ATP の終了コードのみでは証明を受理しない。一階自動推論の導入によって信頼基盤を拡大しない設計。

## Optional notes and sources:

- Source: doc/design/architecture/en/08.reasoning\_boundary.md; Białystok 資料 Story 4; 本資料 Backup 5 に evidence の中身。

## 6.2 - ATP の活用: 探索・検査・再利用



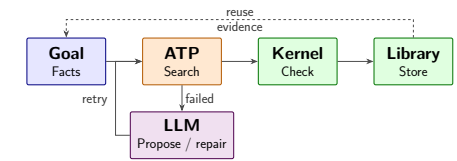
- ゴールと前提 → ATP 探索 → kernel 検査 → ライブラリへ保存・再利用。
- 未解決時は補題・方針を修正して再探索。図は設計意図。

## Mizar Evo の設計指針について

## └ Part 6. Checking And Automation

## └ 6.2 - ATP の活用: 探索・検査・再利用

## 6.2 - ATP の活用: 探索・検査・再利用



- ゴールと前提 → ATP 探索 → kernel 検査 → ライブラリへ保存・再利用。
- 未解決時は補題・方針を修正して再探索。図は設計意図。

## Read aloud:

- **ゴールと前提 → ATP 探索 → kernel 検査 → ライブラリへ保存・再利用。**
- **未解決時は補題・方針を修正して再探索。図は設計意図。**

## Optional notes and sources:

- LLM は提案・修復を支援。受理は evidence の検査で決まり、その仕組みを次のページで示す。
- 現在の ATP 入力引用は前提と局所仮定。ライブラリ全体の前提選択と反復の費用対効果は今後の評価対象。
- Source: doc/spec/en/21.source\_code\_annotation\_and\_atp.md § 21.7.2;  
doc/design/architecture/en/21.ai\_agent\_interface.md。

## 6.3 - Evidence → インスタンス化 → SAT 検査

*evidence* → インスタンス化 → SAT 検査

1. **Evidence**: 元の論理式・明示的な代入・来歴・対象ゴールを保持。
2. **Kernel**: 対応関係と代入を検査し、論理式のインスタンスを生成。
3. **SAT**: インスタンスとゴールの否定を符号化。信頼できる検査器で UNSAT を確認した場合に受理。

## Mizar Evo の設計指針について

## └ Part 6. Checking And Automation

## └ 6.3 - Evidence → インスタンス化 → SAT 検査

6.3 - Evidence → インスタンス化 → SAT 検査

evidence → インスタンス化 → SAT 検査

1. **Evidence**: 元の論理式・明示的な代入・来歴・対象ゴールを保持。
2. **Kernel**: 対応関係と代入を検査し、論理式のインスタンスを生成。
3. **SAT**: インスタンスとゴールの否定を符号化。信頼できる検査器で UNSAT を確認した場合に受理。

## Read aloud:

- **evidence → インスタンス化 → SAT 検査**
- **Evidence**: 元の論理式・明示的な代入・来歴・対象ゴールを保持。
- **Kernel**: 対応関係と代入を検査し、論理式のインスタンスを生成。
- **SAT**: インスタンスとゴールの否定を符号化。信頼できる検査器で UNSAT を確認した場合に受理。

## Optional notes and sources:

- UNSAT により、前提からゴールが従うことを確認。インスタンスと SAT 問題は kernel 自身が導出する。
- バックエンドのトレースと終了コードは診断用。詳細な項目は Backup 5。
- Source: doc/design/architecture/en/08.reasoning\_boundary.md, 15.kernel\_certificate\_format.md.

## 仕様と実装の状況（2026 年 10 月）

- 仕様: 24 章と付録。英語が正典。
- main branch に実装済み: Rust フロントエンド（字句解析・構文解析・構文木）、alpha コーパス上の名前解決・型検査、証明義務の生成・決定的な discharge、ATP 問題の符号化・候補 evidence、SAT に基づく kernel の evidence 検査、キャッシュ・指紋・ビルドスケジューリングの各マイルストーン。
- 進行中: ソースから検証済み成果物までの end-to-end 統合、LSP サーバ、ドキュメント生成。
- 今後の課題: MVM 実行、コード抽出、ライブラリ全体の前提選択、MML 移行。
- 本講演の対象範囲に、外部 ATP を用いた end-to-end の実証結果は含めない。

# Mizar Evo の設計指針について

## └ Status And Roadmap

### └ 仕様と実装の状況 (2026 年 10 月)

#### 仕様と実装の状況 (2026 年 10 月)

- 仕様: 24 章と付録。英語が正典。
- main branch に実装済み: Rust フロントエンド (字句解析・構文解析・構文木)、alpha コーパス上の名前解決・型検査、証明義務の生成・決定的な discharge、ATP 問題の符号化・候補 evidence、SAT に基づく kernel の evidence 検査、キャッシュ・指紋・ビルドスケジューリングの各マイルストーン。
- 進行中: ソースから検証済み成果物までの end-to-end 統合、LSP サーバ、ドキュメント生成。
- 今後の課題: MVM 実行、コード抽出、ライブラリ全体の前提選択、MML 移行。
- 本講演の対象範囲に、外部 ATP を用いた end-to-end の実証結果は含めない。

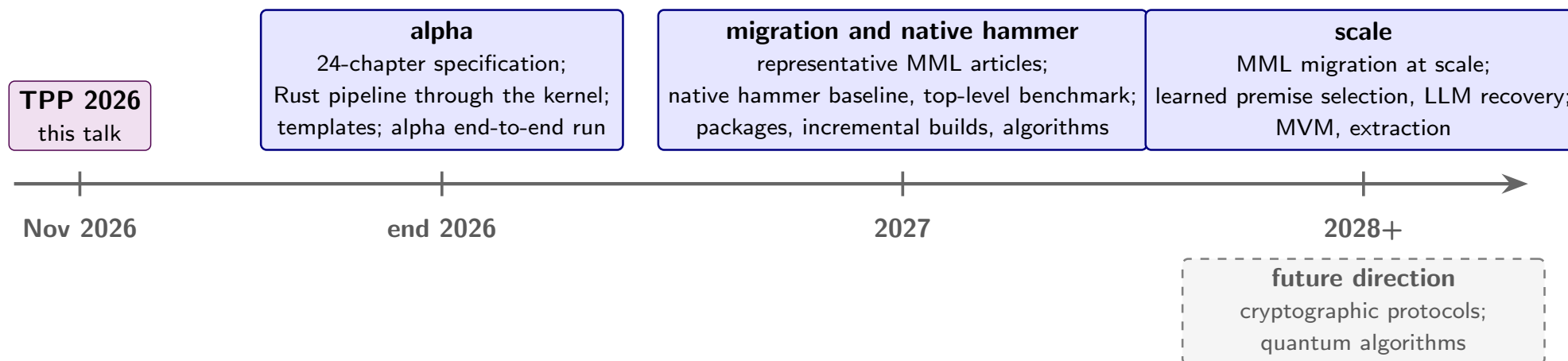
#### Read aloud:

- 仕様: 24 章と付録。英語が正典。
- main branch に実装済み: Rust フロントエンド (字句解析・構文解析・構文木)、alpha コーパス上の名前解決・型検査、証明義務の生成・決定的な discharge、ATP 問題の符号化・候補 evidence、SAT に基づく kernel の evidence 検査、キャッシュ・指紋・ビルドスケジューリングの各マイルストーン。
- 進行中: ソースから検証済み成果物までの end-to-end 統合、LSP サーバ、ドキュメント生成。
- 今後の課題: MVM 実行、コード抽出、ライブラリ全体の前提選択、MML 移行。
- 本講演の対象範囲に、外部 ATP を用いた end-to-end の実証結果は含めない。

#### Optional notes and sources:

- Source: [doc/design/todo.md](#), Crate Status (2026 年 10 月 7 日時点)。講演前に再確認。

# ロードマップ



- 2026 年: 仕様、kernel までの Rust パイプライン、template 処理、alpha の end-to-end 実行の完成。
- 2027 年: 代表的な MML article の移行、native hammer のベースライン構築、MizAR と同じ top-level theorem 単位での評価。
- 2028 年以降: 移行の拡大、学習ベースの前提選択、LLM による失敗回復、MVM・コード抽出。暗号・量子は将来構想。

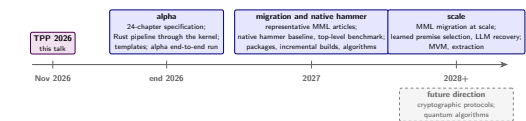
2026-10-09

# Mizar Evo の設計指針について

## └ Status And Roadmap

## └ ロードマップ

### ロードマップ



- 2026 年: 仕様、kernel までの Rust パイプライン、template 処理、alpha の end-to-end 実行の完成。
- 2027 年: 代表的な MML article の移行、native hammer のベースライン構築、MizAR と同じ top-level theorem 単位での評価。
- 2028 年以降: 移行の拡大、学習ベースの前提選択、LLM による失敗回復、MVM・コード抽出。暗号・量子は将来構想。

### Read aloud:

- 2026 年: 仕様、kernel までの Rust パイプライン、template 処理、alpha の end-to-end 実行の完成。
- 2027 年: 代表的な MML article の移行、native hammer のベースライン構築、MizAR と同じ top-level theorem 単位での評価。
- 2028 年以降: 移行の拡大、学習ベースの前提選択、LLM による失敗回復、MVM・コード抽出。暗号・量子は将来構想。

## おわりに

*Keep the foundation small.  
Keep the mathematics readable.  
Modernize everything else.*

- 基盤を小さく保ち、数学的記述の可読性を維持し、周辺基盤を現代化する。
- 議論したい点: MML の移行、記述の利便性、自動化の評価。

2026-10-09

# Mizar Evo の設計指針について

└ Status And Roadmap

└ おわりに

おわりに

*Keep the foundation small.  
Keep the mathematics readable.  
Modernize everything else.*

- 基盤を小さく保ち、数学的記述の可読性を維持し、周辺基盤を現代化する。
- 議論したい点: MML の移行、記述の利便性、自動化の評価。

Read aloud:

- **Keep the foundation small. Keep the mathematics readable. Modernize everything else.**
- **基盤を小さく保ち、数学的記述の可読性を維持し、周辺基盤を現代化する。**
- **議論したい点: MML の移行、記述の利便性、自動化の評価。**

## Backup 1. MizAR 60 の詳細

- **データ:** MPTP で出力した MML 1147、無名の top-level lemma を含む 57,897 件の定理。MizAR 40 と同一版を用いた比較。
- **学習・開発・holdout** を 90:5:5 に分割。holdout 2,896 件中 1,690 件 (58.4%) を hammering mode で証明。ユーザの前提指定なし、420 CPU 秒 (MizAR 40 は約 40.6%)。
- **人または機械がライブラリから前提を選べる条件では 75% 超** (MizAR 40 は 56%)。
- **最高性能の単一手法:** hammering mode で 30 秒 40%。人間の前提指定ありで 120 秒 60%。
- **転移:** 同手法を MML 1382 の新規 242 article、13,370 定理にも適用。
- **手法:** ENIGMA と Deepire で誘導した E と Vampire、学習ベースの前提選択、数百万の ATP 証明で学習するループ。

## Mizar Evo の設計指針について

### └ Backup 1. MizAR 60 の詳細

### └ Backup 1. MizAR 60 の詳細

#### Backup 1. MizAR 60 の詳細

Source: Jakubův et al., MizAR 60 for Mizar 50, ITP 2023.

- データ: MPTP で出力した MML 1147、無名の top-level lemma を含む 57,897 件の定理。MizAR 40 と同一版を用いた比較。
- 学習・開発・holdout を 90:5:5 に分割。holdout 2,896 件中 1,690 件 (58.4%) を hammering mode で証明。ユーザの前提指定なし、420 CPU 秒 (MizAR 40 は約 40.6%)。
- 人または機械がライブラリから前提を選べる条件では 75% 超 (MizAR 40 は 56%)。
- 最高性能の単一手法: hammering mode で 30 秒 40%。人間の前提指定ありで 120 秒 60%。
- 転移: 同手法を MML 1382 の新規 242 article、13,370 定理にも適用。
- 手法: ENIGMA と Deepire で誘導した E と Vampire、学習ベースの前提選択、数百万の ATP 証明で学習するループ。

### Read aloud:

- データ: MPTP で出力した MML 1147、無名の top-level lemma を含む 57,897 件の定理。MizAR 40 と同一版を用いた比較。
- 学習・開発・holdout を 90:5:5 に分割。holdout 2,896 件中 1,690 件 (58.4%) を hammering mode で証明。ユーザの前提指定なし、420 CPU 秒 (MizAR 40 は約 40.6%)。
- 人または機械がライブラリから前提を選べる条件では 75% 超 (MizAR 40 は 56%)。
- 最高性能の単一手法: hammering mode で 30 秒 40%。人間の前提指定ありで 120 秒 60%。
- 転移: 同手法を MML 1382 の新規 242 article、13,370 定理にも適用。
- 手法: ENIGMA と Deepire で誘導した E と Vampire、学習ベースの前提選択、数百万の ATP 証明で学習するループ。

### Optional notes and sources:

- Source: Jakubův et al., MizAR 60 for Mizar 50, ITP 2023.

## Backup 2. Sledgehammer 評価の詳細

- ITP 2022: 50 entry × 100 **ゴール**。MePo、**基準 512 事実、各構成 30 CPU 秒**。**構成は評価集合から事後選択。再構成は評価対象外。**
- PISA: Isabelle2021-1、Sledgehammer **の** timeout 30 **秒**、**ローカル 5 prover、複数設定の成功集合を集約**。Isabelle **内での検証を成功条件とする。**
- **年代が近くても、局所ゴールと定理全体、探索成功と検証済み証明を混同しない。**

## Mizar Evo の設計指針について

## └ Backup 2. Sledgehammer 評価の詳細

## └ Backup 2. Sledgehammer 評価の詳細

## Backup 2. Sledgehammer 評価の詳細

| 評価                      | 対象・方式                                                 | 成功率              |
|-------------------------|-------------------------------------------------------|------------------|
| ITP 2022                | AFP 5,000 ゴール、greedy 16 構成                            | 68.8 %           |
| Magnushammer (2023 年公開) | PISA 1,000 定理、Sledgehammer 学習による前提選択を用いる Magnushammer | 38.3 %<br>59.5 % |

- ITP 2022: 50 entry × 100 ゴール。MePo、基準 512 事実、各構成 30 CPU 秒。構成は評価集合から事後選択。再構成は評価対象外。
- PISA: Isabelle2021-1、Sledgehammer の timeout 30 秒、ローカル 5 prover、複数設定の成功集合を集約。Isabelle 内での検証を成功条件とする。
- 年代が近くても、局所ゴールと定理全体、探索成功と検証済み証明を混同しない。

## Read aloud:

- ITP 2022: 50 entry × 100 ゴール。MePo、基準 512 事実、各構成 30 CPU 秒。構成は評価集合から事後選択。再構成は評価対象外。
- PISA: Isabelle2021-1、Sledgehammer の timeout 30 秒、ローカル 5 prover、複数設定の成功集合を集約。Isabelle 内での検証を成功条件とする。
- 年代が近くても、局所ゴールと定理全体、探索成功と検証済み証明を混同しない。

## Optional notes and sources:

- Source: Desharnais et al., Seventeen Provers Under the Hammer, ITP 2022, sections 5, 5.6, Table 10; Mikuła et al., Magnushammer, arXiv:2303.04488 (2023 年公開), Table 2, Appendix A.4。

## Backup 3. HOL から FOL への符号化と再構成

- 関数適用: 関数変数の適用を `app(F, X)` で表現。定数の適用はカーリー化を維持、または平坦化。
- ラムダ抽象: `lambda lifting` は定義式付きの新しい定数を導入する。コンビネータ変換が代替。
- 型: 多相な HOL の型を `guard`、`tag`、または単相化で符号化。選択は健全性・完全性・`prover` 性能に影響する。
- 真偽値: 論理式中の真偽値を返す項には追加の符号化が必要。
- 再構成: 使用した補題による `metis`、SMT 証明の `smt replay`、生成した `Isar` テキスト。再構成の失敗は別途集計。

## Mizar Evo の設計指針について

## └ Backup 3. HOL から FOL への符号化と再構成

## └ Backup 3. HOL から FOL への符号化と再構成

## Backup 3. HOL から FOL への符号化と再構成

- 関数適用: 関数変数の適用を  $\text{app}(F, X)$  で表現。定数の適用はカーリー化を維持、または平坦化。
- ラムダ抽象:  $\lambda$  lifting は定義式付きの新しい定数を導入する。コンビネータ変換が代替。
- 型: 多相な HOL の型を  $\text{guard}$ 、 $\text{tag}$ 、または単相化で符号化。選択は健全性・完全性・ $\text{prover}$  性能に影響する。
- 真偽値: 論理式中の真偽値を返す項には追加の符号化が必要。
- 再構成: 使用した補題による  $\text{metis}$ 、SMT 証明の  $\text{smt replay}$ 、生成した  $\text{Isar}$  テキスト。再構成の失敗は別途集計。

Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

## Read aloud:

- 関数適用: 関数変数の適用を  $\text{app}(F, X)$  で表現。定数の適用はカーリー化を維持、または平坦化。
- ラムダ抽象:  $\lambda$  lifting は定義式付きの新しい定数を導入する。コンビネータ変換が代替。
- 型: 多相な HOL の型を  $\text{guard}$ 、 $\text{tag}$ 、または単相化で符号化。選択は健全性・完全性・ $\text{prover}$  性能に影響する。
- 真偽値: 論理式中の真偽値を返す項には追加の符号化が必要。
- 再構成: 使用した補題による  $\text{metis}$ 、SMT 証明の  $\text{smt replay}$ 、生成した  $\text{Isar}$  テキスト。再構成の失敗は別途集計。

## Optional notes and sources:

- Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

## Backup 4. Template のインスタンス化: 一つの証明、多くのビュー

### specification example

```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

### specification example

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]
let R be commutative Ring; PermProduct[R qua AddMagma] :: additive view
```

- 環から Magma への継承経路は二つ。qua によるビューの選択が、記法を決定する。

## Mizar Evo の設計指針について

└ Backup 4. Template のインスタンス化: 一つの証明、多くのビュー

└ Backup 4. Template のインスタンス化: 一つの証明、多くのビュー

### Read aloud:

- 有界な型パラメータと汎用的な定理.
- インスタンス化.
- 環から Magma への継承経路は二つ。qua によるビューの選択が、記法を決定する。

### Optional notes and sources:

- Source: doc/spec/en/18.templates.md, section 18.2.2.

Backup 4. Template のインスタンス化: 一つの証明、多くのビュー

有界な型パラメータと汎用的な定理 specification example (証明は省略)

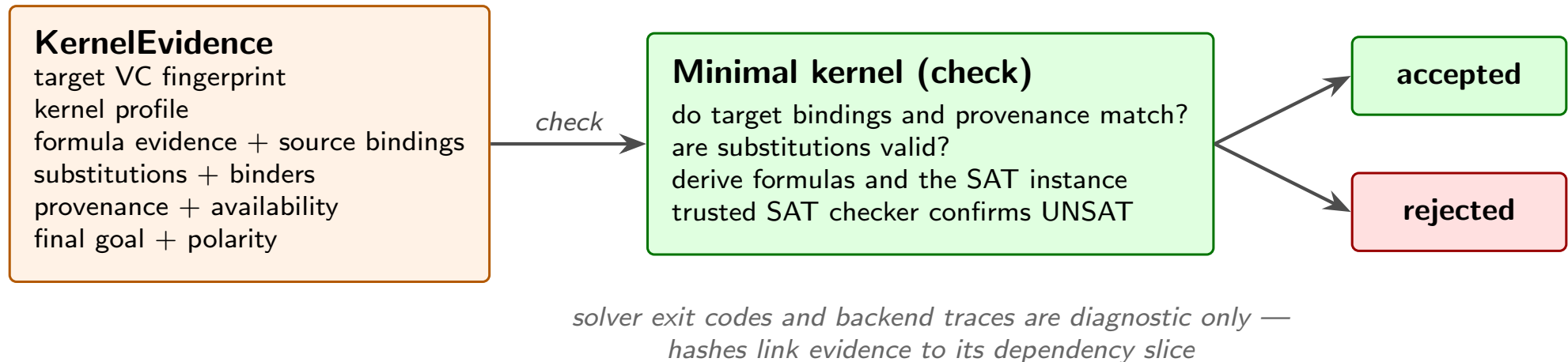
```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

インスタンス化 specification example (必要な registration がある前提)

```
PermProduct [commutative associative unital AddMagma]
PermProduct [commutative associative unital MulMagma]
let R be commutative Ring; PermProduct[R qua AddMagma] :: additive view
```

- 環から Magma への継承経路は二つ。qua によるビューの選択が、記法を決定する。

## Backup 5. Kernel evidence と信頼できる SAT 検査



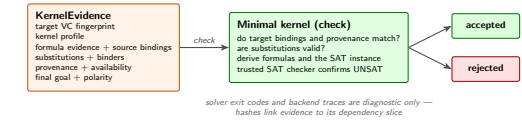
- evidence は元の論理式・明示的な代入・来歴・対象とゴールの対応を保持。kernel はこれらを検査し、決定的な SAT 問題を生成。信頼できるプロセス内 SAT 検査器で UNSAT を確認する。
- バックエンドの証明トレース、SMT の証明オブジェクト、ログ、終了コードは診断用のみ。

## Mizar Evo の設計指針について

### └ Backup 5. Kernel evidence と信頼できる SAT 検査

### └ Backup 5. Kernel evidence と信頼できる SAT 検査

Backup 5. Kernel evidence と信頼できる SAT 検査



- evidence は元の論理式・明示的な代入・来歴・対象とゴールの対応を保持。kernel はこれらを検査し、決定的な SAT 問題を生成。信頼できるプロセス内 SAT 検査器で UNSAT を確認する。
- バックエンドの証明トレース、SMT の証明オブジェクト、ログ、終了コードは診断用のみ。

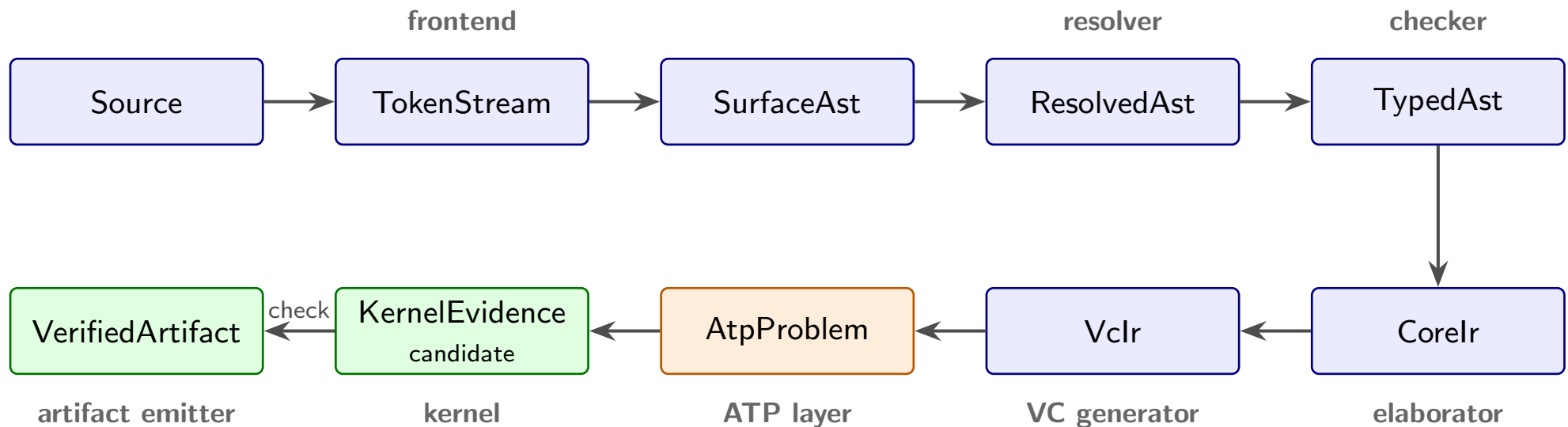
### Read aloud:

- evidence は元の論理式・明示的な代入・来歴・対象とゴールの対応を保持。kernel はこれらを検査し、決定的な SAT 問題を生成。信頼できるプロセス内 SAT 検査器で UNSAT を確認する。
- バックエンドの証明トレース、SMT の証明オブジェクト、ログ、終了コードは診断用のみ。

### Optional notes and sources:

- Source: `doc/design/architecture/en/08.reasoning_boundary.md`, `15.kernel_certificate_format.md`.

## Backup 6. ATP の中心経路



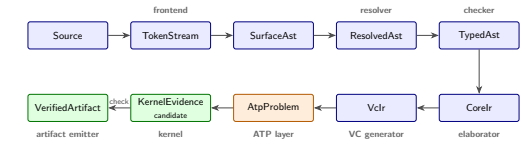
- 決定的な discharge 後の未解決義務のみを ATP に送る。前段の discharge にも再検査可能な evidence が必要。
- 各境界で、情報の管理主体・記録する成果物・変更後の再検査範囲を定める。

## Mizar Evo の設計指針について

## └ Backup 6. ATP の中心経路

## └ Backup 6. ATP の中心経路

Backup 6. ATP の中心経路



- 決定的な discharge 後の未解決義務のみを ATP に送る。前段の discharge にも再検査可能な evidence が必要。
- 各境界で、情報の管理主体・記録する成果物・変更後の再検査範囲を定める。

## Read aloud:

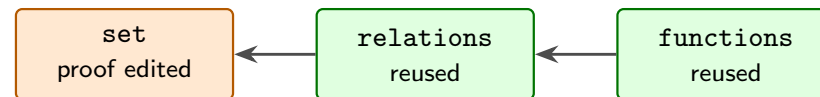
- 決定的な discharge 後の未解決義務のみを ATP に送る。前段の discharge にも再検査可能な evidence が必要。
- 各境界で、情報の管理主体・記録する成果物・変更後の再検査範囲を定める。

## Optional notes and sources:

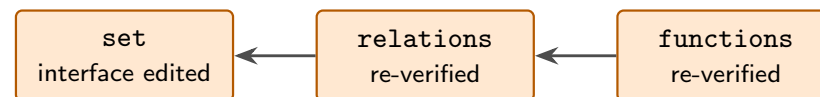
- Source: [doc/design/architecture/en/00.pipeline\\_overview.md](doc/design/architecture/en/00.pipeline_overview.md); Białystok 資料 Part 10。

## Backup 7. 差分検証

theorem proof edit:  
public statement + accepted status unchanged — importers reuse



interface change:  
the dependency cone re-verifies



*reuse requires all relevant keys: source/dependency slices, package/lockfile, toolchain/schema, registration/cluster, verifier policy, evidence/witnesses; missing data is a cache miss — a clean build must reproduce every acceptance*

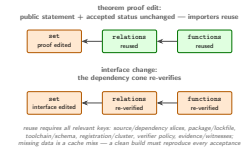
- 証明本体の編集時、公開された主張と受理状態が不変なら、依存側の再ビルドは不要。インタフェース変更時は依存コーンを再検証。
- キャッシュ再利用には、関連キーの完全一致が必要。データ欠落時はキャッシュミス。再利用自体は証明の正当性を保証せず、クリーンビルドで全受理結果を再現する。

## Mizar Evo の設計指針について

## └ Backup 7. 差分検証

## └ Backup 7. 差分検証

## Backup 7. 差分検証



- 証明本体の編集時、公開された主張と受理状態が不変なら、依存側の再ビルドは不要。インタフェース変更時は依存コーンを再検証。
- キャッシュ再利用には、関連キーの完全一致が必要。データ欠落時はキャッシュミス。再利用自体は証明の正当性を保証せず、クリーンビルドで全受理結果を再現する。

## Read aloud:

- 証明本体の編集時、公開された主張と受理状態が不変なら、依存側の再ビルドは不要。インタフェース変更時は依存コーンを再検証。
- キャッシュ再利用には、関連キーの完全一致が必要。データ欠落時はキャッシュミス。再利用自体は証明の正当性を保証せず、クリーンビルドで全受理結果を再現する。

## Optional notes and sources:

- Source: `doc/design/architecture/en/11.artifact_and_incremental_build.md`, `18.dependency_fingerprint.md`; Białystok 資料 Story 5。

## Backup 8. Package manifest

### specification example

```
[package]
name      = "algebra"
version   = "2.3.1"
edition   = "2025"

[dependencies]
mml_core  = "^1.0"
topology  = { version = "^0.9", features = ["metric"] }
```

- 再現可能なビルドの要件: 固定したソース・lockfile・ツールチェーン・検査器設定・決定的な ATP evidence。
- バージョン管理された再利用により、article 集合間の手作業コピーを解消。集約モジュールで、一分野を一つの import として公開できる。

## Mizar Evo の設計指針について

└ Backup 8. Package manifest

└ Backup 8. Package manifest

Backup 8. Package manifest

Mizar Evo **specification example**

```
[package]
name = "algebra"
version = "2.3.1"
edition = "2025"

[dependencies]
mml_core = "~1.0"
topology = { version = "~0.9", features = ["metric"] }
```

- 再現可能なビルドの要件: 固定したソース・lockfile・ツールチェーン・検査器設定・決定的な ATP evidence。
- バージョン管理された再利用により、article 集合間の手作業コピーを解消。集約モジュールで、一分野を一つの import として公開できる。

Source: doc/spec/en/23.package\_management\_and\_build\_system.md; Białystok 資料 Story 1。

### Read aloud:

- 再現可能なビルドの要件: 固定したソース・lockfile・ツールチェーン・検査器設定・決定的な ATP evidence。
- バージョン管理された再利用により、article 集合間の手作業コピーを解消。集約モジュールで、一分野を一つの import として公開できる。

### Optional notes and sources:

- Source: doc/spec/en/23.package\_management\_and\_build\_system.md; Białystok 資料 Story 1。

## Backup 9. 45 分版での追加

1. 単一のゴールで HOL から FOL への関数の符号化を説明 (app、部分適用、lambda lifting)。
2. Mizar の関数表現: Function of  $X, Y$ 、 $f.x$ 、集合論的対象としての soft type。
3. Template: 加法と乗法のビューを持つ PermProduct[T] (Backup 4)。
4. Algorithm: ユークリッドの互除法、by computation、関数子への昇格。
5. Structure とビュー: AddMagma、MulMagma、Magma (Białystok Story 2)。
6. 現代的な基盤: mizar.pkg、namespace、指紋グラフ (Backup 7-8)。
7. 検査: evidence のインスタンス化と SAT 検査 (本編 6.3、詳細は Backup 5)。

## Mizar Evo の設計指針について

└ Backup 9. 45 分版での追加

└ Backup 9. 45 分版での追加

### Backup 9. 45 分版での追加

時間に応じ、以下の具体例を追加:

1. 単一のゴールで HOL から FOL への関数の符号化を説明 (app、部分適用、lambda lifting)。
2. Mizar の関数表現: Function of  $X, Y$ ,  $f.x$ 、集合論的対象としての soft type。
3. Template: 加法と乗法のビューを持つ PermProduct[T] (Backup 4)。
4. Algorithm: ユークリッドの互除法、by computation、関数子への昇格。
5. Structure とビュー: AddMagma, MulMagma, Magma (Białystok Story 2)。
6. 現代的な基盤: mizar.pkg、namespace、指紋グラフ (Backup 7-8)。
7. 検査: evidence のインスタンス化と SAT 検査 (本編 6.3、詳細は Backup 5)。

### Read aloud:

- 単一のゴールで HOL から FOL への関数の符号化を説明 (app、部分適用、lambda lifting)。
- Mizar の関数表現: Function of  $X, Y$ ,  $f.x$ 、集合論的対象としての soft type。
- Template: 加法と乗法のビューを持つ PermProduct[T] (Backup 4)。
- Algorithm: ユークリッドの互除法、by computation、関数子への昇格。
- Structure とビュー: AddMagma、MulMagma、Magma (Białystok Story 2)。
- 現代的な基盤: mizar.pkg、namespace、指紋グラフ (Backup 7-8)。
- 検査: evidence のインスタンス化と SAT 検査 (本編 6.3、詳細は Backup 5)。

## Backup 10. 出典と帰属

- FUNCT\_1: [https://mizar.uwb.edu.pl/version/current/mml/funct\\_1.miz](https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz)
- FUNCT\_2: [https://mizar.uwb.edu.pl/version/current/mml/funct\\_2.miz](https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz)
- **MML のテキストは GPL-3.0-or-later / CC-BY-SA-3.0-or-later。 article 名・URL・行番号を発表者ノートに記載。**
- **ベンチマークの数値は Backup 1-2 と references.bib を参照。最終版の作成前に、出版社の情報で書誌事項を確認。**

# Mizar Evo の設計指針について

## └ Backup 10. 出典と帰属

## └ Backup 10. 出典と帰属

### Backup 10. 出典と帰属

#### 本講演における MML の原文引用:

| 目的                                       | 出典                       | 行       | 使用箇所      |
|------------------------------------------|--------------------------|---------|-----------|
| 関数適用 $f.x$                               | <code>funct_1.miz</code> | 138-140 | Frame 1.2 |
| soft type としての<br><i>Function of X,Y</i> | <code>funct_2.miz</code> | 87-90   | Frame 1.2 |

#### Source URLs:

- FUNCT\_1: [https://mizar.uwb.edu.pl/version/current/mml/funct\\_1.miz](https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz)
- FUNCT\_2: [https://mizar.uwb.edu.pl/version/current/mml/funct\\_2.miz](https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz)

#### 帰属についての注記:

- MML のテキストは GPL-3.0-or-later / CC-BY-SA-3.0-or-later。article 名・URL・行番号を発表者ノートに記載。
- ベンチマークの数値は Backup 1-2 と `references.bib` を参照。最終版の作成前に、出版社の情報で書誌事項を確認。

## Read aloud:

- FUNCT\_1: [https://mizar.uwb.edu.pl/version/current/mml/funct\\_1.miz](https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz)
- FUNCT\_2: [https://mizar.uwb.edu.pl/version/current/mml/funct\\_2.miz](https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz)
- MML のテキストは GPL-3.0-or-later / CC-BY-SA-3.0-or-later。article 名・URL・行番号を発表者ノートに記載。
- ベンチマークの数値は Backup 1-2 と `references.bib` を参照。最終版の作成前に、出版社の情報で書誌事項を確認。

## Backup 11. 評価条件の比較

- 評価対象・前提選択・時間予算・構成選択が異なる。成功率だけでは設計上の優劣を判定できない。

## Mizar Evo の設計指針について

### └ Backup 11. 評価条件の比較

### └ Backup 11. 評価条件の比較

Backup 11. 評価条件の比較

|          | Mizar 60 (ITP 2023)                           | Sledgehammer / AFP (ITP 2022)         |
|----------|-----------------------------------------------|---------------------------------------|
| 報告された成功率 | 58.4% (1,690 / 2,896)                         | 68.8% (3,440 / 5,000)                 |
| 評価対象     | MML 1147 の定理・補題。全 57,897 件のうち bailout 2,896 件 | AFP の 50 entry から選んだ箇所ごと 5,000 件      |
| 前提選択     | ライブラリ全体から学習的に選択                               | MePo。基準 512 事実。構成により増減                |
| 時間予算     | ポートフォリオ合計 420 CPU 秒                           | greedy 16 構成 × 各 30 CPU 秒 = 480 CPU 秒 |
| 成功の判定    | ATP 証明の発見 (hammering mode)                    | 外部 prover の証明発見。Isabelle 内の再構成は評価対象外  |

• 評価対象・前提選択・時間予算・構成選択が異なる。成功率だけでは設計上の優劣を判定できない。

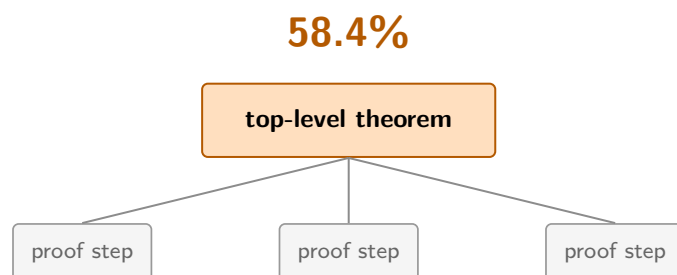
### Read aloud:

- 評価対象・前提選択・時間予算・構成選択が異なる。成功率だけでは設計上の優劣を判定できない。

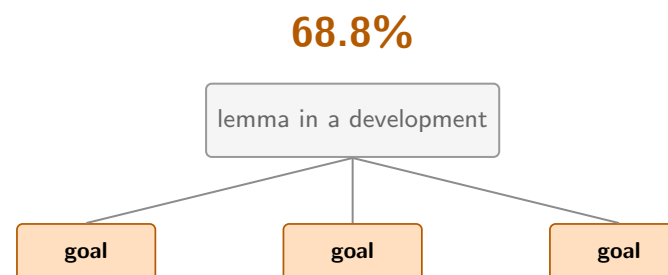
### Optional notes and sources:

- 「ハンマー」は goal を自動証明器に送る道具。「前提」は prover が使ってよい事実。
- Source: Jakubův et al. 2023, sections 6.2, 6.5; Desharnais et al. 2022, sections 5, 5.6, Table 10 (greedy 構成は同じ評価集合から事後選択)。
- 公表年と実験時期は異なる。Mizar の主要実験は 2020 – 2021 年。AFP 評価は 2022 年 1 月の Isabelle と 2021 年 12 月の AFP を使用。
- MizAR の 75% は、人または機械がライブラリから前提を選んだ条件。質問用にとっておく。

## Backup 12. 評価単位: 定理全体と局所ゴール



Mizar 60 (ITP 2023) counts *whole*  
*top-level theorems and lemmas*  
2,896 holdout problems from 57,897 in  
MML 1147; learned premises; 420 CPU s



Sledgehammer / AFP (ITP 2022)  
counts *proof goals as they arise*  
5,000 goals from 50 entries; greedy 16 con-  
figurations × 30 CPU s; no reconstruction

*Different units, premise selection, and budgets.*

- MizAR: 著者による前提指定なしに、ライブラリから定理全体を自動証明できるか。
- AFP 研究: 証明中の局所ゴールを、その文脈で自動証明できるか。多くは人手で記述された証明の内部に位置する。
- いずれも妥当な評価であるが、対象とする課題は異なる。

# Mizar Evo の設計指針について

## └ Backup 12. 評価単位: 定理全体と局所ゴール

## └ Backup 12. 評価単位: 定理全体と局所ゴール

### Read aloud:

- MizAR: 著者による前提指定なしに、ライブラリから定理全体を自動証明できるか。
- AFP 研究: 証明中の局所ゴールを、その文脈で自動証明できるか。多くは人手で記述された証明の内部に位置する。
- いずれも妥当な評価であるが、対象とする課題は異なる。

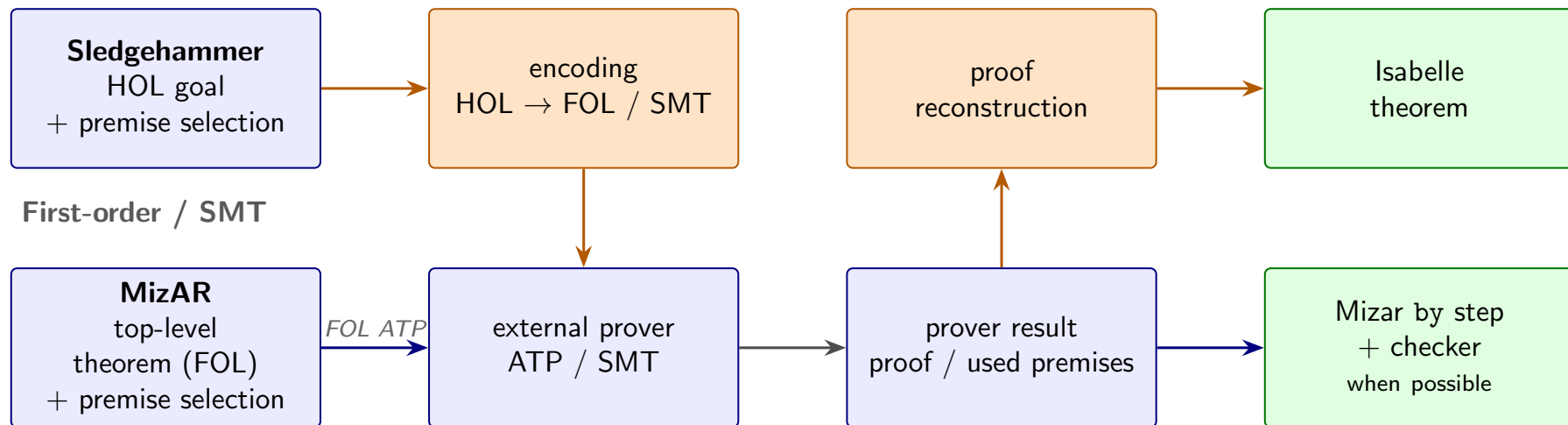
Backup 12. 評価単位: 定理全体と局所ゴール



- MizAR: 著者による前提指定なしに、ライブラリから定理全体を自動証明できるか。
- AFP 研究: 証明中の局所ゴールを、その文脈で自動証明できるか。多くは人手で記述された証明の内部に位置する。
- いずれも妥当な評価であるが、対象とする課題は異なる。

## Backup 13. 一階 ATP への二つの接続経路

Higher-order (Isabelle/HOL)



*Mizar Evo: kernel evidence checking replaces the last step (later slide).*

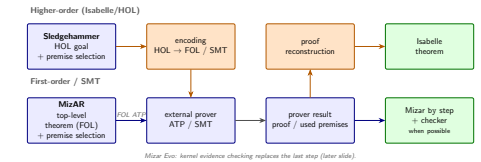
- Sledgehammer の一階・SMT 経路: 高階のゴールを翻訳し、得られた証明を Isabelle 内で再構成する。
- MizAR: 一階・集合論の問題を ATP へ送る。

## Mizar Evo の設計指針について

## └ Backup 13. 一階 ATP への二つの接続経路

## └ Backup 13. 一階 ATP への二つの接続経路

Backup 13. 一階 ATP への二つの接続経路



- Sledgehammer の一階・SMT 経路: 高階のゴールを翻訳し、得られた証明を Isabelle 内で再構成する。
- MizAR: 一階・集合論の問題を ATP へ送る。

## Read aloud:

- Sledgehammer の一階・SMT 経路: 高階のゴールを翻訳し、得られた証明を Isabelle 内で再構成する。
- MizAR: 一階・集合論の問題を ATP へ送る。

## Optional notes and sources:

- どちらも既存システムの説明 (Blanchette, Kaliszyk, Paulson, Urban 2016; Jakubův et al. 2023)。オレンジの箱が翻訳と再構成の層。
- 図は一階・SMT 接続の概形。ITP 2022 の評価は、高階形式を直接扱う prover も含む。
- MizAR の見出しの数字は ATP 証明を数える。Mizar checker は、推論が検査器の強さの範囲なら、得られた by ステップを再検査する。

## Backup 14. 高階論理における関数

sketch

```
f :: 'a => 'b      x :: 'a  
  
P (f x)
```

- HOL: 関数型と関数適用を論理に組み込む。
- 関数を引数・戻り値とする関数、部分適用、ラムダ抽象を直接記述できる。
- 高階の構造を用いた簡潔な数学的記述が可能。
- E や Vampire の一階推論を利用するには、高階構造を一階表現へ符号化する必要がある。

## Mizar Evo の設計指針について

### └ Backup 14. 高階論理における関数

### └ Backup 14. 高階論理における関数

#### Backup 14. 高階論理における関数

HOL の文の概形 **sketch** (Isabelle/HOL 風の記法)

$f :: 'a \rightarrow 'b$        $x :: 'a$

$P (f\ x)$

- HOL: 関数型と関数適用を論理に組み込む。
- 関数を引数・戻り値とする関数、部分適用、ラムダ抽象を直接記述できる。
- 高階の構造を用いた簡潔な数学的記述が可能。
- E や Vampire の一階推論を利用するには、高階構造を一階表現へ符号化する必要がある。

### Read aloud:

- HOL の文の概形.
- **HOL: 関数型と関数適用を論理に組み込む。**
- 関数を引数・戻り値とする関数、部分適用、ラムダ抽象を直接記述できる。
- **高階の構造を用いた簡潔な数学的記述が可能。**
- **E や Vampire の一階推論を利用するには、高階構造を一階表現へ符号化する必要がある。**

## Backup 15. HOL の符号化と再構成

### sketch

```
F X          -> app(F, X)
(%x. t) ...   -> fresh constant + defining axioms   (lambda lifting)
polymorphic types -> type guards or type tags
Boolean-valued terms -> extra encoding
```

- 関数変数の適用は `app` で明示。ラムダ抽象は `lambda lifting` またはコンビネータで、型は `guard` または `tag` で符号化する。
- ATP の証明を Isabelle 内で再構成 (reconstruction) : `metis`、`smt` の `replay`、生成した `Isar` テキストを利用。
- 高階 ITP と一階 ATP の接続を実現する技術。
- 接続の仕組みを示す説明であり、実測上の性能の優劣を示すものではない。

## Mizar Evo の設計指針について

## └ Backup 15. HOL の符号化と再構成

## └ Backup 15. HOL の符号化と再構成

## Backup 15. HOL の符号化と再構成

## 符号化の手順の概形

| 符号化の手段             | 対象                  |
|--------------------|---------------------|
| 関数変数の適用            | app                 |
| ラムダ抽象              | lambda lifting      |
| 型                  | guard または tag       |
| ATP の証明            | metis, smt の replay |
| 高階 ITP と一階 ATP の接続 | metis, smt の replay |
| 接続の仕組み             | metis, smt の replay |

- 関数変数の適用は app で明示。ラムダ抽象は lambda lifting またはコンビネータで、型は guard または tag で符号化する。
- ATP の証明を Isabelle 内で再構成 (reconstruction) : metis、smt の replay、生成した Isar テキストを利用。
- 高階 ITP と一階 ATP の接続を実現する技術。
- 接続の仕組みを示す説明であり、実測上の性能の優劣を示すものではない。

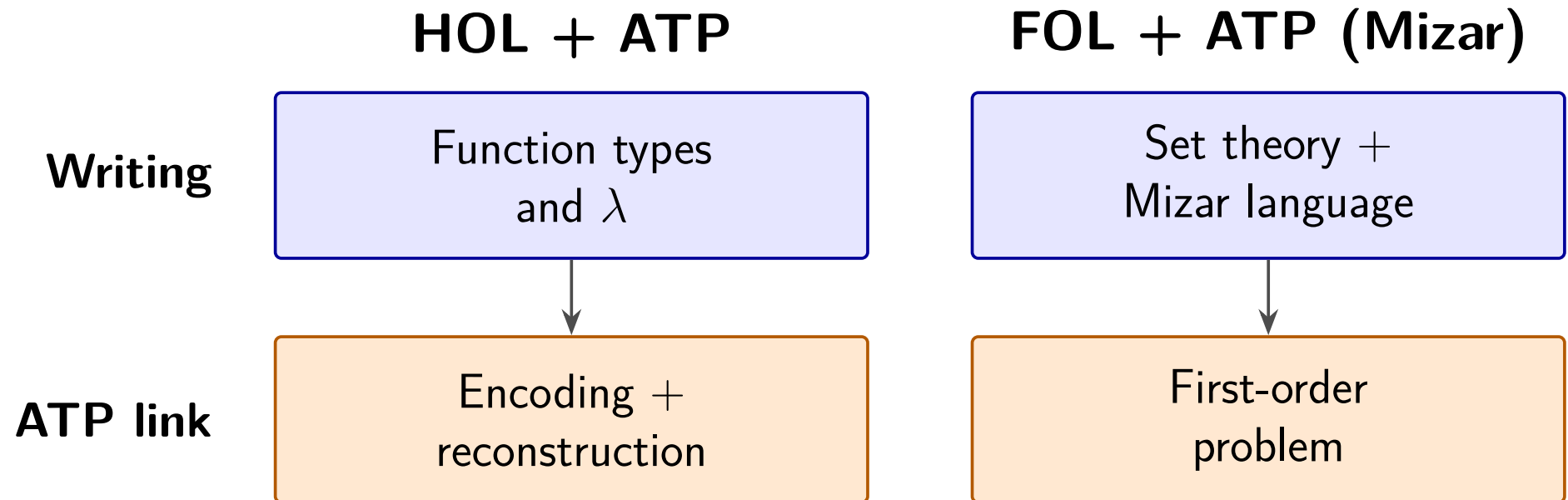
## Read aloud:

- 符号化の手順の概形。
- 関数変数の適用は app で明示。ラムダ抽象は lambda lifting またはコンビネータで、型は guard または tag で符号化する。
- ATP の証明を Isabelle 内で再構成 (reconstruction) : metis、smt の replay、生成した Isar テキストを利用。
- 高階 ITP と一階 ATP の接続を実現する技術。
- 接続の仕組みを示す説明であり、実測上の性能の優劣を示すものではない。

## Optional notes and sources:

- Source: Meng and Paulson 2008 (高階節から一階節への翻訳) ; Blanchette, Böhme, Popescu, Smallbone 2016 (型の符号化) ; Blanchette, Kaliszyk, Paulson, Urban 2016 (サーベイ) ; Schurr, Fleury, Desharnais 2021 (再構成)。
- Sledgehammer が実際に使う符号化は prover とオプションで変わる。この枚は概形。

## Backup 16. 複雑さを担う層の違い



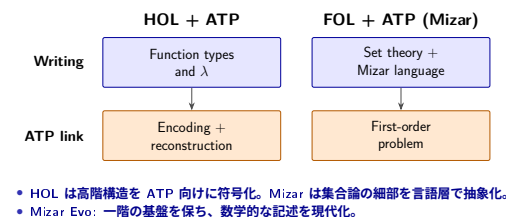
- HOL は高階構造を ATP 向けに符号化。Mizar は集合論の細部を言語層で抽象化。
- Mizar Evo: 一階の基盤を保ち、数学的な記述を現代化。

## Mizar Evo の設計指針について

└ Backup 16. 複雑さを担う層の違い

└ Backup 16. 複雑さを担う層の違い

Backup 16. 複雑さを担う層の違い



## Read aloud:

- HOL は高階構造を ATP 向けに符号化。Mizar は集合論の細部を言語層で抽象化。
- Mizar Evo: 一階の基盤を保ち、数学的な記述を現代化。

## Optional notes and sources:

- HOL と FOL では複雑さを担う層が異なる。HOL は関数型・ラムダを論理に組み込み、一階 ATP との接続では符号化と再構成を行う。
- 集合論の直接記述に伴う対・定義域などの細部は、Mizar の soft type、mode、attribute、registration、scheme で抽象化する。

## Backup 17. 設計上のトレードオフ

- 表現と検査の経路を比較した表であり、実測上の性能優位性を示すものではない。

# Mizar Evo の設計指針について

## └ Backup 17. 設計上のトレードオフ

## └ Backup 17. 設計上のトレードオフ

### Read aloud:

- 表現と検査の経路を比較した表であり、実測上の性能優位性を示すものではない。

Backup 17. 設計上のトレードオフ

|        | HOL ITP + ATP     | FOL ITP + ATP        |
|--------|-------------------|----------------------|
| 記述     | 高階機能による簡潔な記述      | 直接記述すると冗長            |
| 関数     | 基本的な高階の対象         | 一階の集合論的対象            |
| ATP 接続 | 高階構造の一階 + SMT 符号化 | 一階関数の生成              |
| 再構成    | Isabelle 内での再構成   | 対応可能な Mizar ステップの再検査 |
| 言語層の役割 | HOL の抽象           | 一階の細部の抽象化            |

- 表現と検査の経路を比較した表であり、実測上の性能優位性を示すものではない。